Demostración de Concurrencia: El Problema de la Última Botella

1. Contexto y Nivel de Aislamiento Utilizado En PostgreSQL, el nivel de aislamiento transaccional por defecto es **READ COMMITTED (Lectura Confirmada)**. Bajo este nivel de aislamiento:

- Cada sentencia SELECT dentro de una transacción ve únicamente los datos que fueron confirmados (COMMIT) antes de que comenzara esa sentencia SELECT individual (una "foto" o *snapshot* de la base al inicio de la sentencia).
- No existen lecturas sucias (*Dirty Reads*), pero si dos transacciones leen el mismo dato simultáneamente sin bloquearlo y luego deciden modificarlo en base a lo que leyeron, se produce la anomalía de **Actualización Perdida (*Lost Update*) / Condición de Carrera (*Race Condition*)**.

Para demostrar este comportamiento sobre la regla de negocio **RN-09** (*«No se puede vender más de lo que hay en stock de la sala. El stock nunca es negativo»*), se configura un escenario inicial donde la **Sala 1** posee en stock_sala **únicamente 1 botella** disponible (cantidad = 1) del **Vino 1**, y dos vendedores (Sesión A y Sesión B) intentan vender esa última botella en simultáneo.

2. Experimento 1: SIN Bloqueo Explícito (Lectura simple con SELECT)

- **Qué ocurre paso a paso:**

1. Tanto la **Sesión A** como la **Sesión B** inician su transacción con BEGIN;.
2. La **Sesión A** ejecuta SELECT cantidad FROM stock_sala WHERE id_sala = 1 AND cod_vino = 1; y lee 1. Como no usó FOR UPDATE, no adquiere ningún bloqueo de escritura sobre la fila.
3. Antes de que la Sesión A confirme su venta, la **Sesión B** ejecuta el mismo SELECT. Como la Sesión A aún no hizo COMMIT, en el nivel READ COMMITTED la Sesión B también lee cantidad = 1 y su lógica de aplicación cree que hay stock disponible.
4. Ambas sesiones creen tener la botella asegurada e intentan descontar el stock. Si la validación se hiciera únicamente leyendo el valor sin bloqueo antes de actualizar (o actualizando un valor calculado en memoria SET cantidad = 0), ambas transacciones venderían la misma botella física dos veces. En nuestra base de datos, cuando la segunda sesión intenta ejecutar UPDATE stock_sala SET cantidad = cantidad - 1, recién en ese último paso choca contra la restricción CHECK (cantidad >= 0) porque la validación inicial sin bloqueo fue engañada por la lectura concurrente.

```
bodegas_atuel=# UPDATE stock_sala SET cantidad = 1 WHERE id_sala = 1 AND cod_vino = 1;
UPDATE 1
bodegas_atuel=# BEGIN;
BEGIN
bodegas_atuel=*# SELECT cantidad FROM stock_sala WHERE id_sala = 1 AND cod_vino = 1;
 cantidad
----------
        1
(1 row)

bodegas_atuel=*#
```

TERMINAL A

```
bodegas_atuel=# BEGIN;
BEGIN
bodegas_atuel=*# SELECT cantidad FROM stock_sala WHERE id_sala = 1 AND cod_vino = 1;
 cantidad
----------
        1
(1 row)

bodegas_atuel=*#
```

TERMINAL B

```
bodegas_atuel=*# UPDATE stock_sala SET cantidad = cantidad - 1 WHERE id_sala = 1 AND cod_vino = 1;
UPDATE 1
bodegas_atuel=*# COMMIT;
COMMIT
bodegas_atuel=#
```

TERMINAL A

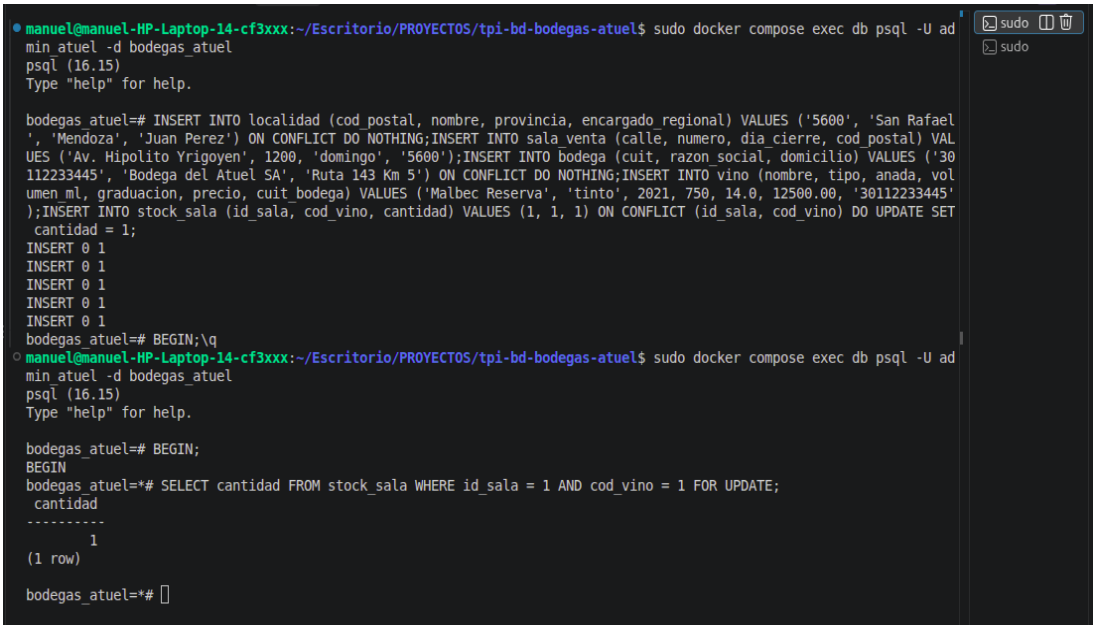
```
bodegas_atuel=*# UPDATE stock_sala SET cantidad = cantidad - 1 WHERE id_sala = 1 AND cod_vino = 1;
ERROR:  new row for relation "stock_sala" violates check constraint "stock_sala_cantidad_check"
DETAIL:  Failing row contains (1, 1, -1).
bodegas_atuel=!#
```

TERMINAL B

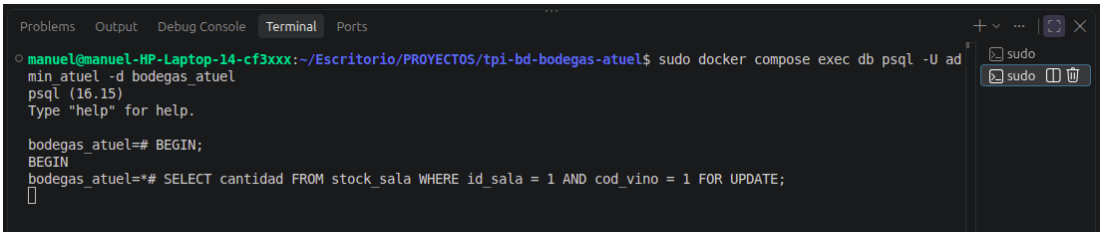
3. Experimento 2: CON Bloqueo (SELECT ... FOR UPDATE)

- **Qué ocurre paso a paso:**

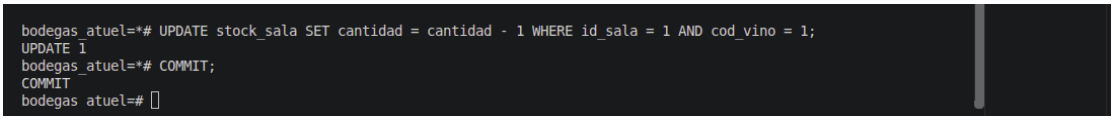
1. La **Sesión A** y la **Sesión B** inician su transacción con BEGIN;.
2. La **Sesión A** ejecuta SELECT cantidad FROM stock_sala WHERE id_sala = 1 AND cod_vino = 1 FOR UPDATE;. Obtiene cantidad = 1 y, en el mismo acto, coloca un **bloqueo exclusivo a nivel de fila (*row-level lock*)** sobre ese registro de stock_sala.
3. Simultáneamente, la **Sesión B** ejecuta la misma sentencia SELECT ... FOR UPDATE. Al encontrar la fila bloqueada por la Sesión A, **PostgreSQL deja a la Sesión B en espera (bloqueada)** hasta que la Sesión A termine su transacción (COMMIT o ROLLBACK).
4. La **Sesión A** descuenta la botella (UPDATE stock_sala SET cantidad = cantidad - 1 ...) dejando el stock en 0, y ejecuta COMMIT;.
5. En el instante exacto en que la Sesión A hace COMMIT, PostgreSQL libera el bloqueo y despierta a la **Sesión B**. En el nivel READ COMMITTED, SELECT ... FOR UPDATE reevalúa la fila con la versión recién confirmada por A y **le devuelve a la Sesión B cantidad = 0**.
6. Al recibir 0, la Sesión B detecta inmediatamente que ya no hay stock antes de intentar vender y rechaza la operación de forma segura (**RN-09**).



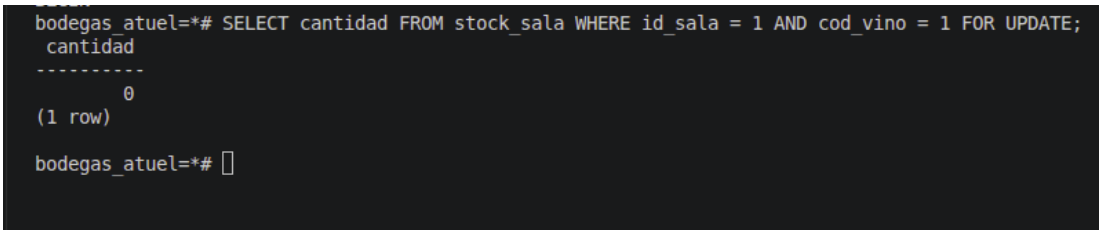
TERMINAL A



TERMINAL B



TERMINAL A



TERMINAL B